Daily Scholar Papers Report — 2026-07-28

2026-07-28

Daily Scholar Papers Report — 2026-07-28

Download PDF

Window covered: 2026-07-27 → 2026-07-28 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

The Google Scholar channel woke up after two silent days and delivered four candidates across two alert threads; three survived screening. The self-curated queue contributed nothing new — both emails it returned were the same two papers already covered in yesterday's digest, still inside Gmail's rolling 24-hour window, and were dropped at intake.

ICAE-Bench (Peng, Huang, Zhang, Xu, Xiao, Hong, Lo, Qiu, Cao, He, Cao — Fudan / Meituan / Singapore Management University / Shanghai Innovation Institute, arXiv preprint, CC BY 4.0) is the substantial entry. It targets a gap that has been widening quietly: coding-agent benchmarks still hand the agent a fully specified task, while the workflows people actually run start from a vague product request and resolve the details through conversation. ICAE-Bench starts each of its **480 tasks across 12 programming languages** from a deliberately fuzzy product requirement — averaging **276 tokens**, against **6,764 tokens** for the complete specification it was derived from — and makes the missing detail recoverable only by asking. The questions go to an automated **User Agent**, a router backed by per-task records that can reveal a hidden constraint but cannot invent a new requirement or leak implementation artefacts, under a budget of **16 queries**. Because every task is derived from a real open-source repository whose original tests were refactored into standardised black-box cases, the resulting repository can be scored by execution rather than by a judge model.

The headline result is bleak in a useful way: the best agent, Claude-Opus-4.8, passes **38.2 %** of cases, GPT-5.5 follows at **37.2 %**, and only **1–13 of 480 repositories per model** are clean on every case. But the finding with the longest shelf life is the paper's separation of two failure modes that aggregate scores conflate — a **requirement-access gap** between the fuzzy brief and what clarification can recover, and an **information-to-execution gap** between recovered requirements and a working repository. The evidence that the second gap dominates is sharp. Giving GPT-5.5 an interaction budget it uses well produces the **best constraint coverage of any model on both splits (73.7 %)** and still leaves it one point *behind* Claude-Opus-4.8 overall. Raising the budget from 16 to 24 questions lifts coverage from 63.8 % to 71.4 % while overall pass rate *falls* from 37.4 % to 34.4 %. Conversely, the single intervention that helps most is not more information but more *actionable* information: dropping ready-to-run test files into the workspace lifts pass rate **from 37.4 % to 61.8 %** while constraint coverage slightly declines. Agents are not failing because they cannot find out what to build; they are failing to carry what they learned through to the artefact.

Two further results deserve to escape the paper. Swapping the agent framework while holding the model fixed moves scores **5.5 to 21.8 points**, enough to reorder the leaderboard — GPT-5.5 drops from 53.3 % to 31.5 % under OpenHands and falls behind Claude-Opus-4.8, which means any published coding-agent ranking is reporting a model-harness pair rather than a model. And a richer execution image, with more dependencies pre-installed, *lowers* pass rate from 37.4 % to 28.4 % while inflating generated repositories to **674 % of the reference line count** — abundance invites over-building.

Two shorter entries round out the window. **ShieldMCP** (Han, Ma, Liu, Niu, Lo — Singapore Management University, ACISP 2026) applies hardware TEE protection to MCP servers at *tool-function* granularity rather than wrapping the whole server, using an LLM agent to generate unit tests and dynamic analysis to trim each function before enclaving it: **97 % average test coverage** and **91 % average code-size reduction**. **The Dark Side of Upgrades** (Wang, He, Wu, Zhou, Wu, Wang — Zhejiang University / City University of Hong Kong, ACISP 2026) contributes the first large-scale dataset of smart-contract upgrade behaviour — **83,085 upgraded contracts, 20,902 upgrade chains** — plus a taxonomy of eight upgrade risk types grounded in 37 real incidents, of which **four types are found to be publicly overlooked and unmitigated**.

Outstanding: 1 · **Keep:** 2 · **Borderline High-Priority:** 0

Highlighted Papers

#	Title	Authors	Venue	Link
1	ICAE-Bench: Evaluating Coding Agents as Interactive Project Builders	Z. Peng, D. Huang, C. Zhang, C. Xu, C. Xiao, S. Hong, D. Lo, L. Qiu, X. Cao, J. He, Y. Cao	**arXiv preprint** (cs.AI), July 2026	arXiv:2607.21217
2	Shielding MCP Tool: A Secure Execution Framework Using TEE and Automated Trimming	R. Han, C. Ma, Y. Liu, Y. Niu, D. Lo	**ACISP 2026**, LNCS 16792, pp. 401–421	doi:10.1007/978-981-92-3012-9_17
3	The Dark Side of Upgrades: Uncovering Insecurity in Smart Contract Upgrades	D. Wang, J. He, S. Wu, Y. Zhou, L. Wu, C. Wang	**ACISP 2026**	arXiv:2508.02145

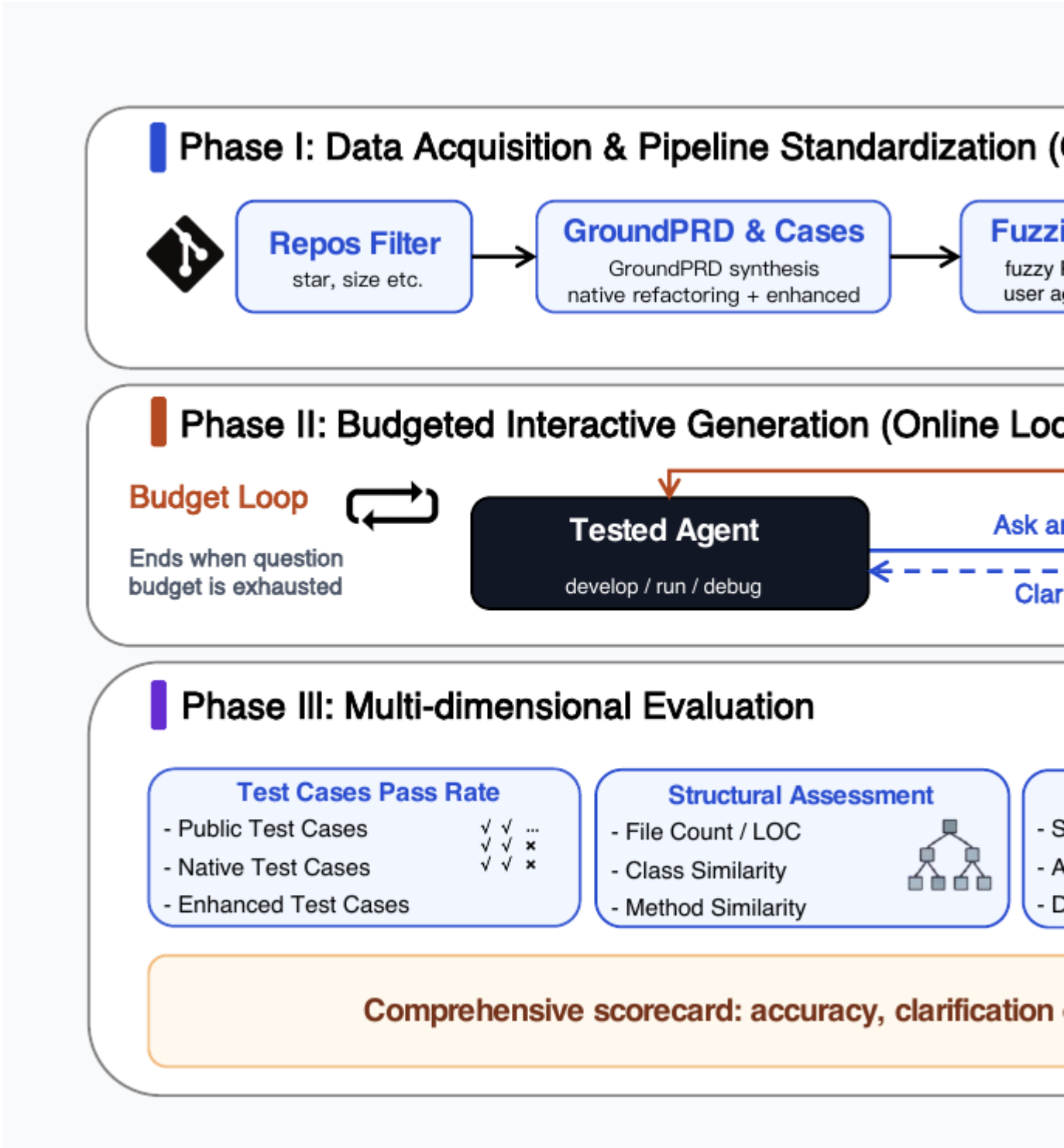
1.1 · Coding agent evaluation · 480 tasks in 12 languages that start from a deliberately vague brief — the best agent passes 38.2 % of cases, and the model with the best requirement recovery still finishes second, separating the requirement-access gap from the information-to-execution gap.

Authors. Zhongyuan Peng (Fudan University; Meituan Group)*, Dan Huang (Singapore Management University)*, Chuyu Zhang (Meituan Group), Caijun Xu (Fudan University; Shanghai Innovation Institute), Changyi Xiao (Fudan University), Shibo Hong (Fudan University), David Lo (Singapore Management

University), Lin Qiu (Meituan Group), Xuezhi Cao (Meituan Group), Jiyuan He (Meituan Group)†, Yixin Cao (Fudan University; Shanghai Innovation Institute)†. *(* equal contribution; † corresponding authors.)*

Venue. Preprint. `arXiv:2607.21217v1 [cs.AI]`, 23 July 2026, 23 pages. Abstract page: https://arxiv.org/abs/2607.21217.

Problem. Coding agents are increasingly asked to turn incomplete product intent into working software — planning, asking clarifying questions, using tools, debugging, and building a repository — but benchmarks have not followed. Function-level suites such as HumanEval and MBPP and repository-editing suites such as SWE-bench and RepoBench all presuppose that the target software context already exists and that the goal is static, explicit, or discoverable without asking anyone. The from-scratch benchmarks that move closer still supply detailed requirements, fixed scaffolds, or language-specific constraints; the nearest prior work asks agents to rebuild a program from an executable plus documentation, but the target behaviour remains exposed through a static interface. Work that does study clarification evaluates it at function level or inside an existing repository. The gap ICAE-Bench occupies is clarification during **0-to-1 repository construction**, where a missing requirement does not just change a return value — it changes the architecture, the public interface, the dependency set, the test protocol, and the edge-case contract.



ICAE-Bench framework in three phases. Phase I, offline data acquisition: repository filtering, GroundPRD and test-case synthesis, fuzzification into a fuzzy PRD plus user agent data, and packaging into an ultimate image with the reference implementation removed. Phase II, a budgeted online loop in which the tested agent develops, runs and debugs while asking the User Agent about ambiguous requirements and receiving clarifications without source leakage, ending when the question budget is exhausted. Phase III, multi-dimensional evaluation across test-case pass rate, structural assessment, agentic evaluation, and interaction quality, combined into a comprehensive scorecard.

Figure 2 from Peng, Huang, Zhang, Xu, Xiao, Hong, Lo, Qiu, Cao, He & Cao, "ICAE-Bench: Evaluating Coding Agents as Interactive Project Builders," arXiv:2607.21217. Reproduced under CC BY 4.0.

Task formalisation. The paper's one numbered equation defines a task as a five-tuple (Eq. 1):

$$\mathcal{T} = (D_f, E, P, U, B)$$

where D_f is the fuzzy PRD and E the ultimate-image execution environment; the Public set P contains illustrative Native cases recoverable through clarification, while B contains all authoritative Native and Enhanced cases; User Agent Data U stores the omitted constraints, the complete input/output content of P, and the records that ground clarification responses. The agent observes only (D_f, E), queries the User Agent backed by U under a fixed budget, and produces a repository evaluated against B. Two further definitions are given inline rather than as numbered equations: the hidden set is Hidden = (Native ∪ Enhanced) \ Public, and constraint coverage is $|H|/|C|$ over hidden constraint identifiers C and those matched at least once H. There are no theorems.

Construction pipeline. Five stages. Real repositories are admitted only if their entire original test suite passes in a controlled Docker environment. Those tests are then transformed into standardised black-box JSON cases — inputs and expected outputs produced by a dispatcher run against the reference implementation, with language-specific exception details normalised into repository-level error categories — forming the **Native** suite; a case is accepted only after passing against the reference. **GroundPRD**, the complete specification, is then fuzzified: constraints spanning API commitments, edge-case behaviour and architectural requirements are stripped out, each assigned an identifier with trigger phrases and a grounded response, producing the fuzzy PRD plus the User Agent Data. Finally the task is packaged into an **ultimate image** built on an official language base image, with the reference code, original tests and construction artefacts removed. A verification pass has a frontier model reimplement each task from GroundPRD until the cases agree, and a second model judge the fuzzified PRD against GroundPRD.

The scale asymmetry is the point of the design: the fuzzy brief averages **276 tokens** while the specification it hides averages **6,764**, and the User Agent Data holding the difference averages **7,091**.

Leakage control operates at three boundaries: the User Agent never sees reference source, original tests, repository identity, or hidden evaluation cases; the coding agent sees only the router's natural-language reply, never the underlying records or the internal match log; and each agent develops in an isolated container with no authoritative cases, which are injected only into a fresh evaluation container after generation. Scoring replays the delivered repository in that fresh container, so anything the agent did transiently during development counts only if it survives into the artefact.

Metrics. Four families, deliberately non-redundant. *Functional correctness* is split by source (Native, transformed from real tests, vs Enhanced, synthesised for robustness) and by visibility (Public, exposed in the brief, vs Hidden), with Overall the case-level pass rate across Native and Enhanced. *Agentic evaluation* uses a critic model over the generated and reference source trees for semantic similarity, API similarity and design quality. *Structural assessment* compares file-count and LOC ratios plus class, method and namespace similarity, to expose under-building, over-building, or the collapse of a multi-module project into a monolithic adapter — explicitly without rewarding literal reproduction. *Interaction quality* comes from the router logs: constraint coverage, fallback rate, and budget usage, macro-averaged at task level so constraint-heavy tasks do not dominate.

Headline numbers. Six models under Claude Code, full 480-task split:

Model	Overall	Public	Native	Enhanced	Constraint cov.	Fallback
Claude-Opus-4.8	**38.2**	48.5	41.4	35.5	69.6	21.6
GPT-5.5	37.2	50.3	42.0	32.8	**73.7**	20.4
Gemini-3.1-Pro	27.0	37.0	30.7	23.5	56.3	28.1
GLM-5.1	26.6	36.8	30.0	23.7	63.9	28.1
Claude-Sonnet-4.6	21.8	29.0	24.1	19.4	57.7	24.2
MiniMax-M2.5	0.8	1.5	1.2	0.6	44.3	28.5

- **Only 1–13 of 480 repositories per model are clean** on every evaluated case — the repository-level bar is far stricter than the case-level rate.
- A persistent **Public–Enhanced gap** across all models: reproducing the visible examples does not yield robust behaviour.
- **Test scaffolding is the strongest single intervention:** ready-to-run Public case files raise Overall from **37.4 % to 61.8 %** — while constraint coverage *falls* from 63.8 % to 61.2 %. The treatment changes how actionable the information is, not how much of it there is.
- **Framework swap moves 5.5–21.8 points** and reorders the ranking: under OpenHands, GPT-5.5 falls 53.3 % → 31.5 % and drops behind Claude-Opus-4.8 (48.2 % → 42.7 %).
- **Budget is non-monotonic:** 8 / 16 / 24 questions give **22.9 % / 37.4 % / 34.4 %** Overall, while coverage climbs 57.9 % → 71.4 % and fallback rate rises. Fuzzy L1 contains slightly more than 20 ambiguity points per task.
- **A richer environment hurts:** the fuller ultimate image lowers Overall from 37.4 % to 28.4 % while expanding output to **191 % of reference file count and 674 % of reference LOC**.
- **Thinking helps materially:** enabling it lifts Gemini-3.1-Pro from 26.7 % to 36.6 %.
- **Best-coverage ≠ best-outcome:** the User Agent backbone producing the *highest* constraint coverage (75.2–75.8 %) yields the *lowest* pass rate.
- Split rankings agree only moderately (Spearman's ρ = 0.71 between the full and Lite splits).

Reliability of the subjective metrics. The critic scores were validated against three annotators over 200 repositories, giving Pearson 0.372 (semantic), 0.381 (API) and 0.471 (design), with intraclass correlations of 0.22–0.37. The authors describe this as positive but moderate and position the critic scores as scalable diagnostics complementary to the execution-verified cases rather than interchangeable with them. That framing is right, and it is worth carrying into any downstream use: the pass-rate columns are execution-verified, the similarity and design columns are not, and they should not be read at the same confidence.

Limitations. There is no dedicated limitations section; the caveats are distributed. The authors state that the Lite split "preserves the top capability tier and supports economical ablations, but not definitive model ranking or language-level conclusions"; that the fuzzy L1–L3 levels form "an operational information-exposure hierarchy rather than a universal psychometric scale"; that structural ratios near 100 % indicate similar scale, "not necessarily better quality"; and that runtime and token use "remain weak proxies for correctness." Beyond those, three gaps are worth naming. The benchmark covers no

frontend, visual or multi-modal tasks, which the authors flag as future work. The User Agent is itself a model, so its behaviour is a confound — demonstrated by the backbone ablation, where the most thorough router produced the worst downstream results. And the construction pipeline is verified using frontier models that then appear at the top of the leaderboard; the black-box execution checks carry most of the weight, but the circularity is not discussed.

Closing line (verbatim). “Across 480 multi-language tasks, it measures not just whether the final code works, but whether agents can recover missing requirements and carry them through to a correct final artifact.”

Follow-up pointers. Code and data at https://github.com/ALEX-nlp/ICAE-EVAL. The most portable idea here is not the benchmark but the **fuzzification-with-oracle** construction: take a complete specification, remove constraints in a structured way, and retain each removal as a queryable record with trigger phrases and a grounded answer. That converts any fully-specified benchmark into an interactive one while keeping the ground truth intact, and it generalises well beyond coding — to any task where the realistic setting is underspecified but the evaluable setting requires a fixed answer.

1.2 · MCP / TEE security · Enclaving MCP servers at tool-function granularity instead of wrapping the whole server: 97 % LLM-generated test coverage drives a 91 % average code-size reduction before the trimmed artefact enters the TEE.

Authors. Ruidong Han, Chengyan Ma, Ye Liu, Yuqing Niu, David Lo — all Singapore Management University.

Venue. ACISP 2026 — Australasian Conference on Information Security and Privacy, LNCS vol. 16792, pp. 401–421. Published 18 July 2026. https://doi.org/10.1007/978-981-92-3012-9_17.

Access note. This entry is written from the published abstract and front matter only — the chapter is subscription content and no preprint was located. No formal characterisation, algorithm detail, or evaluation methodology is asserted below, and no figures are reproduced.

Problem. The Model Context Protocol gives LLMs a standard way to call external tools, but the protocol carries no inherent security mechanism. When an MCP server runs in an untrusted environment, the authenticity, integrity and confidentiality of tool execution are all exposed — the host can observe or tamper with whatever the tool does, which for MCP tools means database queries, file reads and writes, and web fetches performed with the user’s authority.

Approach. ShieldMCP places MCP tool execution inside a hardware Trusted Execution Environment. The design decision that distinguishes it is granularity: rather than encapsulating the entire server — which the authors argue would impose performance overhead and an inflated trusted computing base — protection is applied at the level of the individual **tool function**, the unit an LLM actually invokes to perform a specific external action. The pipeline is automated end to end: scan the MCP server for tool functions; generate unit tests for each using an LLM agent; apply dynamic analysis to trim each function’s code down to what those tests exercise; package the reduced artefact into a TEE; and deploy it behind a secure, attested invocation channel.

Headline numbers. The LLM agent achieved **97 % average test coverage**. Trimming reduced code size by **91 % on average** while maintaining functional correctness. The authors report a qualitative analysis confirming that the architecture mitigates the identified authenticity, integrity and confidentiality violations.

Assessment. Function-level partitioning for enclaves is well-trodden ground — the paper's own reference list reaches back through Android application partitioning and Java enclave partitioning frameworks — and the contribution here is the application to MCP plus the automation of the test-generation step. That step is also where the interesting question sits: a 91 % size reduction validated by tests the same pipeline generated is a weaker guarantee than the number suggests, because the residual risk is precisely the behaviour that neither the generated tests nor the dynamic trace exercised. Trimming that removes a rarely-taken error path changes the function's behaviour under exactly the conditions an attacker would try to induce. The TCB-reduction argument is sound in principle and the direction is timely; the strength of the result depends on how adversarial the coverage is, which the abstract cannot settle. Worth a full read if the chapter becomes reachable.

Follow-up pointers. Dataset at https://figshare.com/s/e5b23f83190cd14cc247. The natural companion is the authors' earlier AutoTEE work on automated migration of programs into trusted execution environments, of which this reads as an MCP-targeted specialisation.

1.3 · Smart contract security · 83,085 upgraded contracts and 20,902 upgrade chains, a taxonomy of eight upgrade risks drawn from 37 real incidents, and four risk types that turn out to be publicly overlooked and unmitigated.

Authors. Dingding Wang, Jianting He, Siwei Wu, Yajin Zhou, Lei Wu (Zhejiang University), Cong Wang (City University of Hong Kong).

Venue. ACISP 2026 — Australasian Conference on Information Security and Privacy. Preprint: arXiv:2508.02145.

Access note. Written from the abstract and bibliographic record. The conference version carries the subtitle *"Uncovering Insecurity in Smart Contract Upgrades"*; the preprint reads *"Uncovering Security Risks…"*. No formal characterisation is asserted, and no figures reproduced.

Problem. Upgradeability has become standard practice for deployed smart contracts, letting teams fix bugs and add features after deployment. It also dissolves the immutability guarantee that much of the security reasoning about contracts rests on. Prior work has looked at the security impact of upgrades, but the authors argue it is limited on two axes at once: the range of upgrade behaviours collected, and the identification of what can actually go wrong.

Contribution. Three parts. First, a dataset of **83,085 upgraded contracts and 20,902 upgrade chains** — described as the first large-scale dataset of upgrade behaviour, and used to show both the diversity of those behaviours and gaps in how upgrades are publicly disclosed. Second, a taxonomy of insecurities derived from **37 real-world security incidents**, sorting upgrade risks into **eight types** and offering what the authors present as the first complete view of upgrade-related insecurity. Third, a survey of public awareness and existing mitigations, which finds **four of the eight risk types are overlooked by the public and lack mitigation measures**; a preliminary detection study over the dataset identifies **31,407 related issues**.

Assessment. This is the well-worn but consistently valuable shape of empirical security research: assemble the measurement substrate nobody has assembled, ground a taxonomy in incidents that actually happened rather than in hypothesised attack trees, then use the taxonomy to show that part of the risk surface has no defenders. The four-of-eight overlooked finding is the load-bearing claim and the one a reader should check the methodology behind, since "overlooked by the public" is a judgement about discourse rather than a measured quantity, and the 31,407 issues come from a study the authors

themselves label preliminary. The dataset is the durable contribution — upgrade chains in particular are the right unit of analysis, since the risk is rarely in any single upgrade but in what the sequence permits.

Follow-up pointers. The direct predecessor is the 2024 work on characterising smart-contract upgrades; this extends it from characterisation to insecurity. The transferable idea for non-blockchain readers is treating **the upgrade chain rather than the artefact version as the unit of security analysis** — a framing that applies to any system with a mutable deployed component and a partial public record of its mutations.

Cross-Paper Synthesis

Today's three papers are separated by field — coding-agent evaluation, trusted execution, blockchain measurement — but two threads run through them.

All three are about what happens when a specification is incomplete, and all three locate the danger after the gap is closed rather than at the gap itself. ICAE-Bench's central finding is that agents recover requirements adequately and then fail to carry them into the artefact; the requirement-access gap is real but the information-to-execution gap is where the losses are. ShieldMCP's trimming removes whatever its generated tests did not exercise, which means the specification implicit in those tests becomes the function's new behavioural contract, and any gap between test coverage and real behaviour is silently converted into a change in semantics. The upgrade study's whole subject is what a contract's specification means when it can be replaced, and its most interesting risks are the ones nobody wrote down as risks. In each case the incompleteness is manageable; the failure is in the step that acts on the incomplete information as though it were complete.

Two of the three lean on LLM-generated ground truth, and neither fully reckons with it. ShieldMCP trims against LLM-generated unit tests at 97 % coverage; ICAE-Bench verifies its task artefacts using frontier models and scores three of its metric columns with a critic model whose agreement with human annotators sits at Pearson 0.37–0.47. ICAE-Bench handles this considerably better — it measures the agreement, publishes it, and explicitly demotes the critic scores to diagnostics while keeping the headline metric execution-verified. That is the right pattern, and the contrast makes it legible: when a model generates your oracle, the question is not whether to do it but what you have independently verified underneath it. ICAE-Bench has black-box execution as its floor; a trimming result validated only by generated tests has no comparable floor.

The harness-versus-model result deserves wider circulation. ICAE-Bench's finding that swapping the agent framework moves scores by up to 21.8 points and reorders the leaderboard is a direct challenge to how coding-agent capability is currently reported. Every published number of the form "model X scores Y on benchmark Z" is really a measurement of a model-harness-scaffold triple, and the harness term is large enough here to dominate the model term for adjacent models. That echoes the pattern flagged in the 2026-07-27 synthesis around externally-checkable harnesses — the harness is not neutral infrastructure sitting underneath the measurement, it is part of what is being measured.

On venue signal. The `venue::preprint` posterior at 0.75 auto-promoted ICAE-Bench today, and it happened to agree with the followed-researcher signal, so it changed nothing. With two observations behind it against a 0.65 threshold, it remains too thin to act on independently, and today's window offers no evidence either way — the two ACISP papers arrived through the alert channel, not through a preference route.

Writing & Rationale Insights

Separate the gaps your aggregate score conflates, then show which one dominates. ICAE-Bench's most reusable contribution is not the 480 tasks but the decomposition into a requirement-access gap and an information-to-execution gap, plus the experiment that isolates them: the model with the best constraint coverage finishes second overall, and raising the interaction budget improves coverage while *lowering* pass rate. A single "agent capability" number would have hidden this entirely. When a benchmark measures a pipeline, the useful design question is which stage the losses occur in, and the evaluation should be built to answer it.

A non-monotonic result is worth more than a monotonic one, and should be foregrounded. Three of ICAE-Bench's ablations go the "wrong" way — more questions, a richer environment, and a more thorough clarification router all reduce final correctness. Those are the findings that change what a reader does next, because they falsify the intuition that more information is better. The paper reports them plainly and builds its discussion around them rather than burying them in an appendix. The transferable habit: when an ablation contradicts the obvious prior, that is the headline, not a caveat.

Publish the agreement statistics for your subjective metrics, even when they are unflattering. Pearson 0.37–0.47 and ICC 0.22–0.37 against human annotators is weak, and reporting it invites exactly the criticism that the critic-scored columns are unreliable. Reporting it anyway, and then explicitly repositioning those columns as diagnostics rather than rankings, is what makes the execution-verified columns believable by contrast. A paper that ran the same validation and omitted it would look identical on the surface and be worth considerably less.

Mark the access boundary of your own evidence. Two of today's three entries are written from abstracts, and saying so changes how every claim in them should be read. The same discipline applies inside papers: distinguishing what was measured from what was inferred, and what was verified by execution from what was scored by a model, costs a sentence and prevents a reader from over-crediting the weakest link. Where a paper does not make that distinction, a careful reader has to reconstruct it — which is work the authors could have done once.

Validate identity before excluding on it. Today's screening required checking a smart-contract paper against an author-level exclusion rule, on a byline of `J He, S Wu, L Wu` that superficially resembled the excluded group. Resolving the full author list showed a different institution entirely, and the paper proceeded. Surname-level matching is the wrong granularity for any exclusion or inclusion rule in a field where a handful of surnames cover a large fraction of authors; the check has to reach affiliation before it fires.

Beware the circularity of verifying a benchmark with the models you rank. ICAE-Bench uses frontier models to confirm that each task is solvable and that each fuzzified brief is faithful, and those same models take the top two leaderboard positions. The execution-based checks carry most of the weight so the design is defensible, but it goes undiscussed, and a reader who wants to cite the ranking has to notice it unaided. When the construction loop and the evaluation subjects overlap, saying so explicitly costs a paragraph and pre-empts the strongest available objection.

Sources: Scholar alert — David Lo new articles · Scholar alert — Recommended articles · arXiv:2607.21217 · doi:10.1007/978-981-92-3012-9_17 · arXiv:2508.02145